

Brandon Nguyen, 827813045
COMPE596 - Spring 2026
4/5/2026

COMPE596 P08 - NVIDIA CUDA Discrete Fourier Transform Acceleration

1) Final FFTW_P08.cpp Source

// Main Program

```
#include <iostream>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>
#include <fftw3.h>
#include <cufft.h>
#include <cuda_runtime.h>
#include <chrono>

#include <soundtouch/SoundTouch.h>

#include "WavFile.h"

using namespace std;
using namespace soundtouch;

#define BUFF_SIZE      16384
#define MAX_FREQ       48 //KHz

#define CUDA_CHECK(ans)
{ gpuAssert((ans), __FILE__, __LINE__); }
inline void gpuAssert(cudaError_t code, const char *file, int line,
                     bool abort = true) {
    if (code != cudaSuccess) {
        fprintf(stderr, "GPUassert: %s %s %d\n", cudaGetErrorString(code),
            file, line);
        if (abort)
            exit(code);
    }
}
```

```

int main(int argc, char *argv[]) {

    const char *wavfile;
    if( argc != 2 )
    {
        fprintf(stderr,"usage: %s <input.wav>\n", argv[0]);
        exit(1);
    }
    else
    {
        wavfile = argv[1];
    }

    char *wavfileout = (char *)malloc(strlen(argv[0]) + strlen("_out.wav") + 1);
    char *logfile = (char *)malloc(strlen(argv[0]) + strlen("_out.log") + 1);
    wavfileout = strcat(strcpy(wavfileout, argv[0]),"_out.wav");
    logfile = strcat(strcpy(logfile, argv[0]),"_out.log");

    FILE *log;
    fftw_complex *in, *out, *out2;
    in = (fftw_complex*) fftw_malloc(sizeof(fftw_complex) * BUFF_SIZE);
    out = (fftw_complex*) fftw_malloc(sizeof(fftw_complex) * BUFF_SIZE);
    out2 = (fftw_complex*) fftw_malloc(sizeof(fftw_complex) * BUFF_SIZE);

    fftw_plan plan = fftw_plan_dft_1d(BUFF_SIZE, in, out, FFTW_FORWARD,
    FFTW_ESTIMATE);
    fftw_plan plan2 = fftw_plan_dft_1d(BUFF_SIZE, out, out2, FFTW_BACKWARD,
    FFTW_ESTIMATE);

    cufftHandle cuplan;
    cufftDoubleComplex *d_DoubleComplexData;
    cufftDoubleComplex *h_DoubleComplexData;
    int nx = BUFF_SIZE;

    cudaMalloc((void**)&d_DoubleComplexData, nx * sizeof(cufftDoubleComplex));
    h_DoubleComplexData = (cufftDoubleComplex *)calloc(nx,
    sizeof(cufftDoubleComplex));

    cufftPlan1d(&cuplan, nx, CUFFT_Z2Z, 1);

```

```
SAMPLETYPE sampleBuffer[BUFF_SIZE];
short buffer[BUFF_SIZE];
static float power[MAX_FREQ];
```

```
double cpu_total_time = 0.0;
double gpu_total_time = 0.0;
```

```
WavInFile inFile(wavfile);
printf("SampleRate: %d Hz\n",inFile.getSampleRate());
printf("Number of bits per sample: %d\n",inFile.getNumBits());
printf("Sample data size in bytes: %d\n",inFile.getDataSizeInBytes());
printf("Total number of samples in file: %d\n",inFile.getNumSamples());
printf("Number of bytes per audio sample: %d\n",inFile.getBytesPerSample());
printf("Number of audio channels in the file (1=mono, 2=stereo):
%d\n",inFile.getNumChannels());
printf("Audio file length in milliseconds: %d\n",inFile.getLengthMS());
```

```
WavOutFile
outFile(wavfileout,inFile.getSampleRate(),inFile.getNumBits(),inFile.getNumChannels());
```

```
log = fopen(logfile,"w");
```

```
while (inFile.eof() == 0) {
```

```
    size_t samplesRead = inFile.read(sampleBuffer, BUFF_SIZE);
```

```
    for (int i = 0; i < BUFF_SIZE; i++) {
        in[i][0] = (double) sampleBuffer[i];
        in[i][1] = 0.0;
        out[i][0] = 0.0;
        out[i][1] = 0.0;
    }
```

```
    auto cpu_start = std::chrono::high_resolution_clock::now();
```

```
    fftw_execute(plan);
```

```
    int Fs = inFile.getSampleRate();
    int N = BUFF_SIZE;
```

```

int target_bin = (10000.0 * N) / Fs;

for(int i = target_bin - 10; i <= target_bin + 10; i++) {
    out[i][0] = 0.0;
    out[i][1] = 0.0;
}

int mirror_bin = N - target_bin;

for(int i = mirror_bin - 10; i <= mirror_bin + 10; i++) {
    out[i][0] = 0.0;
    out[i][1] = 0.0;
}

fftw_execute(plan2);

auto cpu_end = std::chrono::high_resolution_clock::now();
cpu_total_time += std::chrono::duration<double, std::milli>(cpu_end -
cpu_start).count();

CUDA_CHECK(cudaMemcpy(d_DoubleComplexData, out,
    nx * sizeof(cufftDoubleComplex),
    cudaMemcpyHostToDevice));

auto gpu_start = std::chrono::high_resolution_clock::now();

cufftExecZ2Z(cuplan, d_DoubleComplexData, d_DoubleComplexData,
CUFFT_FORWARD);
cufftExecZ2Z(cuplan, d_DoubleComplexData, d_DoubleComplexData,
CUFFT_INVERSE);

CUDA_CHECK(cudaDeviceSynchronize());

auto gpu_end = std::chrono::high_resolution_clock::now();
gpu_total_time += std::chrono::duration<double, std::milli>(gpu_end -
gpu_start).count();

CUDA_CHECK(cudaMemcpy(h_DoubleComplexData, d_DoubleComplexData,

```

```
nx * sizeof(cufftDoubleComplex),  
cudaMemcpyDeviceToHost));
```

```
for (int i = 0; i < nx; i++) {  
    h_DoubleComplexData[i].x /= nx;  
    h_DoubleComplexData[i].y /= nx;  
}
```

```
for (size_t i = 0; i < samplesRead; i++) {  
    int re = out[i][0];  
    int im = out[i][1];  
    float magnitude = sqrt(re * re + im * im);  
    float freq = (i + 1) * Fs / samplesRead;  
    int index = freq / 1000;
```

```
    if (index <= MAX_FREQ) {  
        power[index] += magnitude;  
    }  
}
```

```
for (int i = 0; i < BUFF_SIZE; i++) {  
    buffer[i] = h_DoubleComplexData[i].x;  
}
```

```
outFile.write(buffer, BUFF_SIZE);  
}
```

```
for (int i = 0; i < MAX_FREQ; i++)  
    printf("%2d kHz, power: %9.2f\n", i, power[i]);
```

```
printf("\n==== TIMING RESULTS =====\n");  
printf("Total CPU FFTW Time: %f ms\n", cpu_total_time);  
printf("Total GPU cuFFT Time: %f ms\n", gpu_total_time);
```

```
fftw_destroy_plan(plan);  
fftw_destroy_plan(plan2);  
fftw_free(in);  
fftw_free(out);
```

```

fftw_free(out2);

cufftDestroy(cuplan);
cudaFree(d_DoubleComplexData);

fclose(log);
return 0;
}

```

// Terminal Output

```

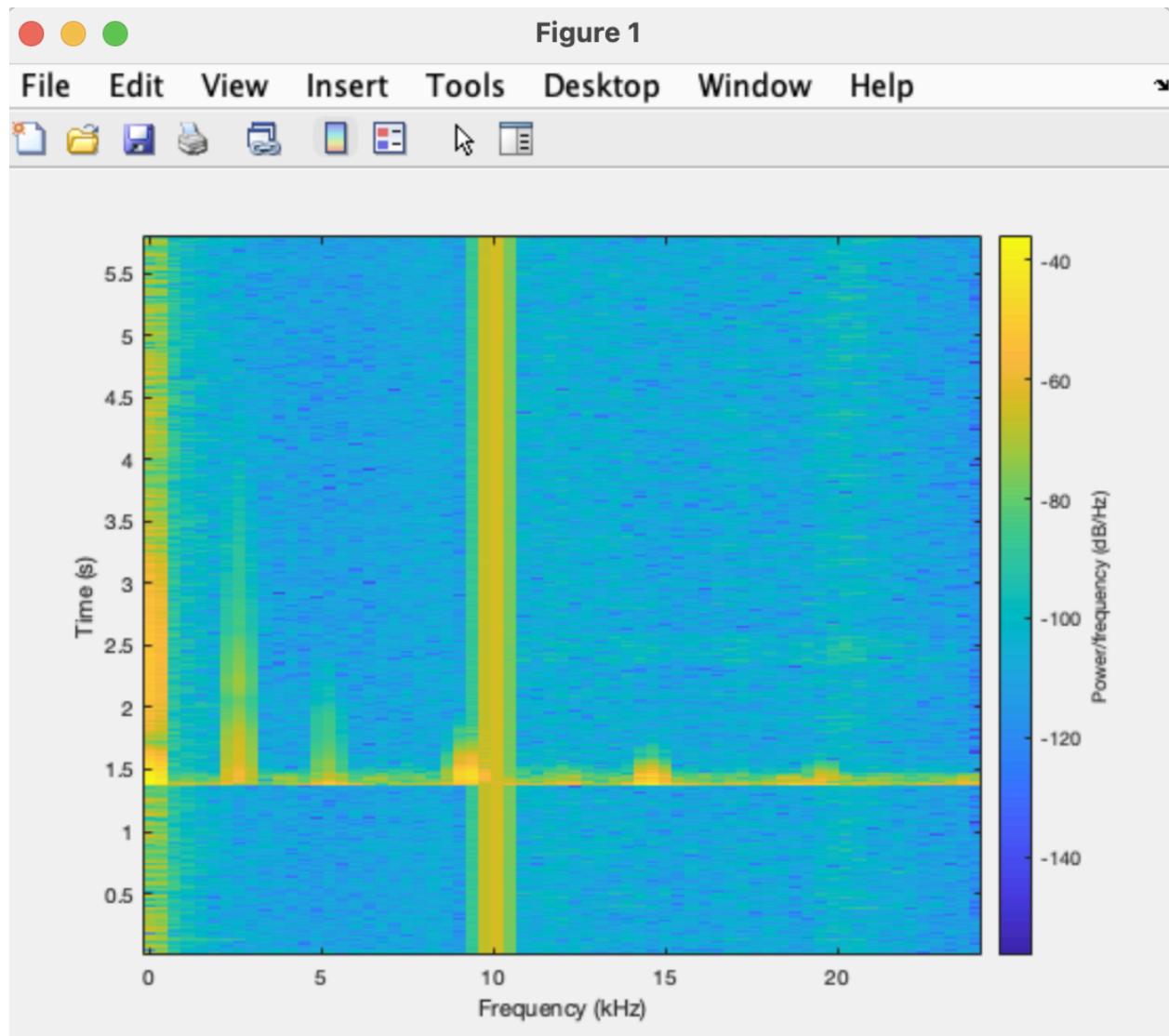
~ -- bnguyen@dgx: ~ -- -zsh

bnguyen@dgx:~$ nvcc -c -I/usr/local/include -I/usr/local/cuda-12.4/targets/x86_64-linux/include FFTW_P08.cpp
bnguyen@dgx:~$ nvcc -c -I/usr/local/include WavFile.cpp
bnguyen@dgx:~$ nvcc FFTW_P08.o WavFile.o -o FFTW_P08 -lfftw3 -L/usr/local/cuda-12.4/targets/x86_64-linux/lib -lcufft -lcudart
bnguyen@dgx:~$ ./FTTW_P08 compe596_mix_48kHz_16bit_mono_Mixdown_1.wav
SampleRate: 48000 Hz
Number of bits per sample: 16
Sample data size in bytes: 557056
Total number of samples in file: 278528
Number of bytes per audio sample: 2
Number of audio channels in the file (1=mono, 2=stereo): 1
Audio file length in milliseconds: 5803
0 kHz, power: 41937.98
1 kHz, power: 1617.48
2 kHz, power: 5124.49
3 kHz, power: 2436.13
4 kHz, power: 2194.00
5 kHz, power: 5674.41
6 kHz, power: 3164.63
7 kHz, power: 2808.81
8 kHz, power: 3253.50
9 kHz, power: 12009.20
10 kHz, power: 6157.28
11 kHz, power: 2499.63
12 kHz, power: 8488.68
13 kHz, power: 2960.70
14 kHz, power: 13521.28
15 kHz, power: 4753.95
16 kHz, power: 2540.77
17 kHz, power: 3441.00
18 kHz, power: 6931.91
19 kHz, power: 5543.96
20 kHz, power: 7855.14
21 kHz, power: 3729.95
22 kHz, power: 1930.38
23 kHz, power: 2728.83
24 kHz, power: 2733.30
25 kHz, power: 1922.00
26 kHz, power: 3710.68
27 kHz, power: 7844.05
28 kHz, power: 5578.57
29 kHz, power: 6907.21
30 kHz, power: 3459.99
31 kHz, power: 2524.56
32 kHz, power: 4763.85
33 kHz, power: 13537.00
34 kHz, power: 2947.04
35 kHz, power: 8419.78
36 kHz, power: 2569.02
37 kHz, power: 6171.85
38 kHz, power: 11976.72
39 kHz, power: 3279.82
40 kHz, power: 2797.55
41 kHz, power: 3163.57
42 kHz, power: 5662.07
43 kHz, power: 2208.06
44 kHz, power: 2410.68
45 kHz, power: 5162.47
46 kHz, power: 1592.29
47 kHz, power: 27454.54
bnguyen@dgx:~$ ls

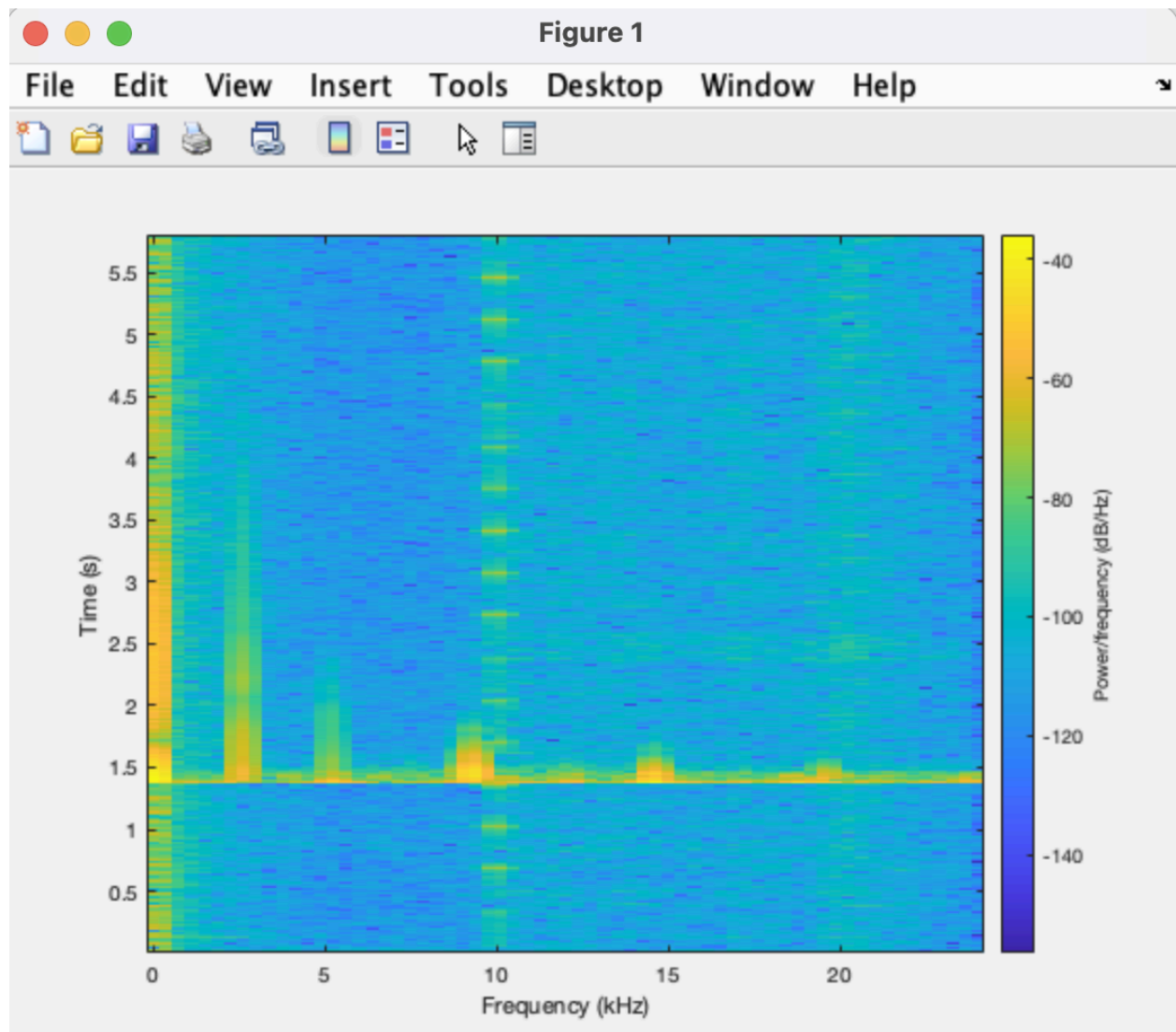
```

2) Spectrogram Plot

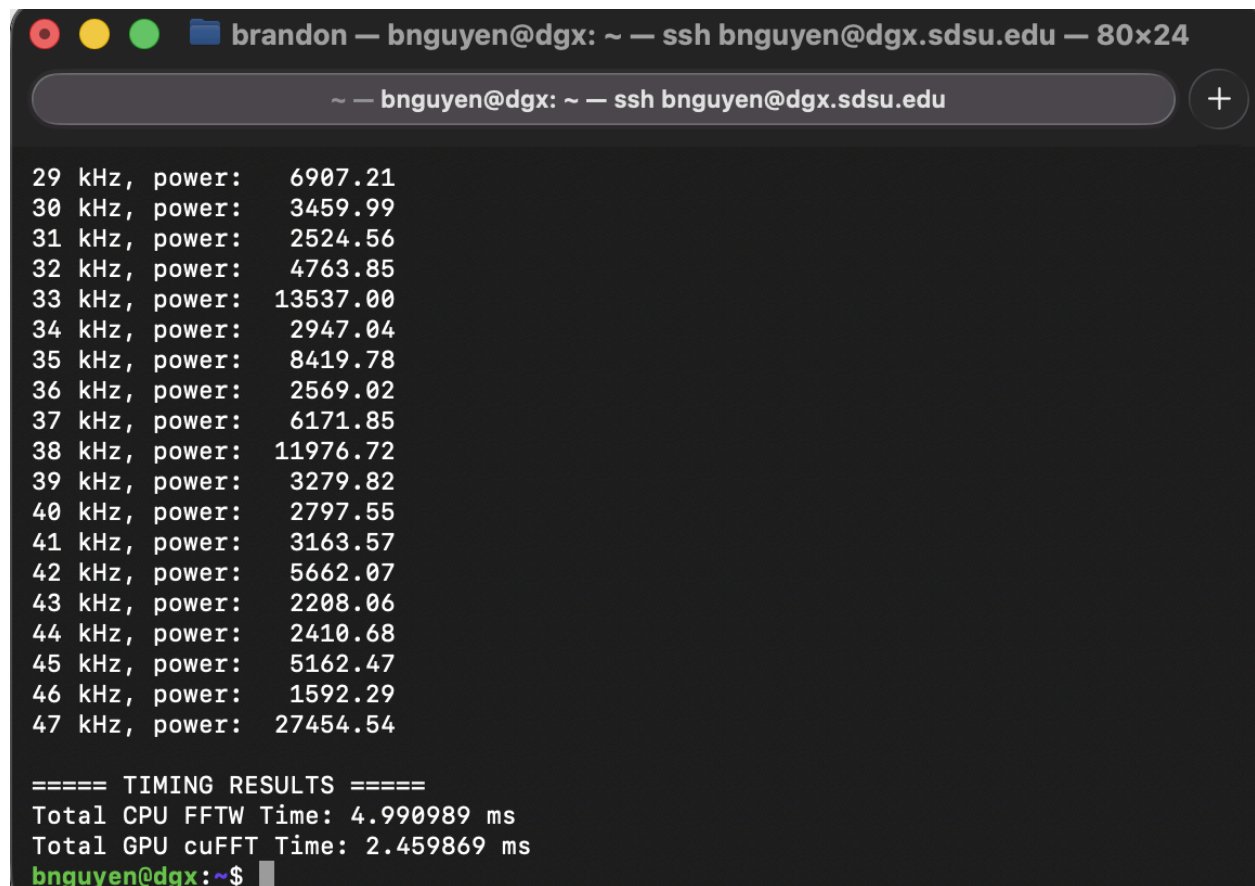
// Unfiltered



// Filtered



3) Elapsed Time Comparison



A terminal window titled "brandon — bnguyen@dgx: ~ — ssh bnguyen@dgx.sdsu.edu — 80x24". The window displays a list of frequency components (kHz) and their corresponding power values. Below this, it shows timing results for CPU FFTW and GPU cuFFT.

Frequency (kHz)	Power
29	6907.21
30	3459.99
31	2524.56
32	4763.85
33	13537.00
34	2947.04
35	8419.78
36	2569.02
37	6171.85
38	11976.72
39	3279.82
40	2797.55
41	3163.57
42	5662.07
43	2208.06
44	2410.68
45	5162.47
46	1592.29
47	27454.54

==== TIMING RESULTS ====
Total CPU FFTW Time: 4.990989 ms
Total GPU cuFFT Time: 2.459869 ms
bnguyen@dgx:~\$

4) Discussion

In P08, we are tasked with observing how to accelerate the discrete Fourier transform on a GPU using the CUDA cuFFT API. First, we build the program FFTW_P08.cpp on dgx.sdsu.edu using the following Make build rules in order to plot and analyze its spectrogram to see that the 10 kHz tone is present in the WAV file. Afterwards, we will be working to edit the FFTW_P08.cpp program to filter out the 10 kHz tone and plot the spectrogram again to confirm the filtering. Lastly, we will implement and measure the elapsed time of performing the combined forward and inverse FFT on a CPU using FFTW and on a GPU using cuFFT. According to the spectrograms above, we were able to filter out most of the 10 kHz tone from the WAV file. Additionally, we see that the elapsed time for GPU using cuFFT (2.46 ms) is faster than the CPU FFTW method (4.99 ms). This is because FFT is a highly parallelizable algorithm, so the GPU is able to compute multiple frequency components simultaneously and cuFFT is heavily optimized for NVIDIA GPUs, therefore the GPU outperforms the CPU here. Overall, the eight programming assignment concludes our work with CUDA programming by showing how it can be applied to WAV audio files and Fourier Transform (FFT).